

# 東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

# Deprecating zone_reclaim_mode

Joshua Hahn (Meta) <joshua.hahnjy@gmail.com>



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

# zone_reclaim_mode?

From Documentation/admin-guide/sysctl/vm.rst:

"... allows someone to set more or less aggressive approaches to reclaim when a zone runs out of memory"

# fallback allocations

2-NUMA system, task is local to Node 0

| Scenario | zone_reclaim_mode = 0 | zone_reclaim_mode > 0 |
|---|---|---|
| Local zone runs out of memory | Fall back to other zones in zonelist and try to fulfill the allocation there | Trigger direct reclaim (shrink_node) |
| Remote node is also full | After iterating through all zones, wake up kswapd, and try again (slowpath). | |

i.e. Try your best to allocate memory locally, by reclaiming the current node.

# My goals for today

1. Are there current usecases for zone_reclaim_mode?

2. Should we deprecate zone_reclaim_mode?

3. What is the path forward?

# My goals for today

NOT to say that zone_reclaim_mode is entirely bad ... but:

For most workloads, it doesn't make sense.

For the workloads where it does make sense, we have better alternatives.

# When is zone_reclaim_mode harmful?

- When cost of direct reclaim is greater than remote access latency
- Systems with mixed workload that require different reclaim behavior
- NUMA nodes with > 1 zones

# Numbers – When zone_reclaim_mode is harmful

Workload: competing anon and file cache memory

- Database workloads (buffer pool + query execution memory)
- Build systems (source code file + compiler memory)
- Compute (input data cache + computation memory)

Microbenchmark: 60G anon + 100G filecache, scanning file sequentially.
2-NUMA node system, each with 125GB memory
average across 5 iterations
numactl –cpunodebind=0 –preferred=0

zone_reclaim_mode=0: 5.56 GB/s
zone_reclaim_mode=7: 0.08 GB/s

# When is zone_reclaim_mode beneficial?

- Latency sensitive workloads that are perfectly sized to the node

# Numbers – When zone_reclaim_mode is beneficial

Workload: memory-bound, latency sensitive workload

- High performance computing

Microbenchmark: 100G filecache + 60G anon, random memory access
2-NUMA node system, each with 125GB memory
average across 5 iterations
numactl –cpunodebind=0 –preferred=0

zone_reclaim_mode=0: 67.86 ns/access
zone_reclaim_mode=7: 56.34 ns/access

# Numbers – Do we have alternatives?

Workload: memory-bound, latency sensitive workload

- High performance computing

Microbenchmark: 100G filecache + 60G anon, random memory access
2-NUMA node system, each with 125GB memory
average across 5 iterations
numactl –cpunodebind=0 –preferred=0

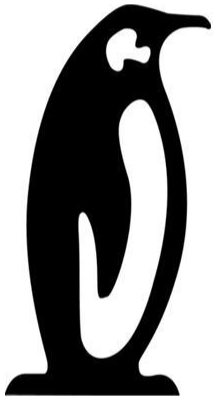
zone_reclaim_mode=0, –preferred=0: 67.86 ns/access
zone_reclaim_mode=0, –membind=0: 52.98 ns/access
zone_reclaim_mode=7, –preferred=0: 56.34 ns/access

# It's a togglable feature. Why not just leave it there?

- Maintenance burden (deprecate 3 sysctls)
- Prevent users from making suboptimal decisions for their system
- Simplify page allocation code

# Discussion

- Did I miss any usecases? Any nuances to the workloads?
- Is NUMA balancing good enough to correct remote memory spillage?
- Should we stop at deprecating the sysctl, or introduce new ones to provide flexibility to the user?
- How to actually deprecate user-facing sysctls?



# 東京 2025
# LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

